Vellore Institute of Technology – School of Computer Science and Engineering
BCSE303L – Operating Systems | Fall Semester 2026-27 | Case Study (CS3)

Adaptive Page Replacement: Selecting a Replacement Policy from Observed Memory-Reference Locality

Batch member	Register number	Contribution (Sec. 6)
Shupraja N	24BCE0560	See Section 6
Aishwarya Vaidyanathan	24BCE2683	See Section 6

Code, workload generators, configuration and raw CSV files: https://github.com/Shupraja18/CS3-Adaptive-page-replacement.git

Abstract

Fixed page-replacement algorithms each win on some reference patterns and lose on others. This study implements FIFO, LRU, Optimal, Clock and LFU, validates them on controlled reference strings, and designs two adaptive policies that change the eviction rule while the program runs: Adaptive-F reads four locality features from a sliding window, and Adaptive-S replays every candidate rule in shadow caches and adopts the best one. Six synthetic reference patterns were tested at three workload sizes (10 000 to 100 000 references, 16 to 64 frames) with ten random seeds each. On a workload whose behaviour changes over time the adaptive policies reduced page faults by roughly 10 to 20 % compared with the best fixed policy (Wilcoxon $p \leq 0.01$), and on stationary workloads they stayed close to the best fixed policy. One clear failure was found: Adaptive-F handles a noisy cyclic loop poorly. The cost of adaptivity, its limits and its threats to validity are discussed.

1. Introduction

Demand paging lets a process run with only part of its address space in physical memory. When a referenced page is absent a page fault occurs, and a replacement algorithm must choose a victim frame. Because a fault costs a disk or SSD access, many orders of magnitude more than a memory access, the choice of victim strongly affects performance. Belady's optimal algorithm (OPT) minimises faults but needs knowledge of future references, so real systems use approximations such as LRU, Clock or frequency-based schemes [1, 3].

No fixed approximation is best for all programs. LRU assumes recent pages will be reused and collapses on a cyclic loop slightly larger than memory; LFU resists one-time scans but keeps stale pages after a program changes phase; FIFO ignores usage altogether and can even show Belady's anomaly [2]. This case study asks: can an adaptive policy detect the current locality behaviour and select or tune a replacement strategy so that it is competitive across patterns? We (i) implement the five required algorithms, (ii) validate them, (iii) design two adaptive mechanisms, and (iv) compare adaptive and fixed policies on page faults, hit ratio and execution time with statistical tests.

2. Methodology

2.1 Simulator design

All online policies share one class that keeps, for every resident page, its load time, last-use time, reference count and reference bit. A policy is just the rule used to pick a victim: FIFO takes the oldest load time, LRU the oldest last use, LFU the lowest count (ties broken by LRU), and Clock sweeps a hand over pages in load order, clearing reference bits and giving a second chance. Because every rule has the metadata it needs at all times, the rule can be switched between two references without any conversion step, which is what the adaptive policies exploit. We also include MRU as an extra baseline and adaptive candidate, because it suits loops. OPT is implemented separately using precomputed next-use positions; as it needs the future, it is a lower bound and never an adaptive candidate. Cold-start faults are counted. Metrics are page faults, hit ratio ($1 - \text{faults}/N$) and wall-clock simulation time.

2.2 Validation on controlled reference strings

Table 1. Validation checks (all passed)

Check	Expected	Obtained	Status
FIFO / LRU / OPT, textbook string [1], 3 frames	15 / 12 / 9	15 / 12 / 9	Pass
FIFO, Belady string, 3 → 4 frames	9 → 10 (anomaly)	9 → 10	Pass
LRU, Belady string, 3 → 4 frames	10 → 8	10 → 8	Pass
OPT, Belady string, 3 → 4 frames	7 → 6	7 → 6	Pass

Check	Expected	Obtained	Status
LRU vs. independent stack-distance LRU (6 patterns)	identical faults	identical	Pass
OPT ≤ every policy; every policy ≥ distinct pages	always true	true	Pass
Frames ≥ distinct pages	faults = distinct pages	equal	Pass

```
[notice] To update, run: C:\Users\shupr\AppData\Local\Programs\Python\Python313\python.exe -m pip install --
PS C:\Users\shupr\Downloads\cs3_adaptive_paging\cs3_adaptive_paging> py tests/test_validation.py
  PASS  FIFO textbook, 3 frames: got 15, expected 15
  PASS  LRU  textbook, 3 frames: got 12, expected 12
  PASS  OPT  textbook, 3 frames: got 9, expected 9
  PASS  FIFO Belady, 3 frames: got 9, expected 9
  PASS  FIFO Belady, 4 frames: got 10, expected 10
  PASS  LRU  Belady, 3 frames: got 10, expected 10
  PASS  LRU  Belady, 4 frames: got 8, expected 8
  PASS  OPT  Belady, 3 frames: got 7, expected 7
  PASS  OPT  Belady, 4 frames: got 6, expected 6
  PASS  property checks on 'random' (OPT minimal, >= cold misses, LRU == stack-distance LRU)
  PASS  property checks on 'zipf' (OPT minimal, >= cold misses, LRU == stack-distance LRU)
  PASS  property checks on 'working_set' (OPT minimal, >= cold misses, LRU == stack-distance LRU)
  PASS  property checks on 'loop' (OPT minimal, >= cold misses, LRU == stack-distance LRU)
  PASS  property checks on 'scan_mix' (OPT minimal, >= cold misses, LRU == stack-distance LRU)
  PASS  property checks on 'phase_mix' (OPT minimal, >= cold misses, LRU == stack-distance LRU)
  PASS  frames >= distinct pages -> faults == distinct pages

  ALL VALIDATION CHECKS PASSED
PS C:\Users\shupr\Downloads\cs3_adaptive_paging\cs3_adaptive_paging>
```

Figure 1. Validation run output.

Clock hand trace

Step	Page	Frames	Reference bits	Result
1	1	1	1	Fault
2	2	1, 2	1, 1	Fault
3	3	1, 2, 3	1, 1, 1	Fault
4	2	1, 2, 3	1, 1, 1	Hit
5	4	2, 3, 4	0, 0, 1	Fault
6	1	3, 4, 1	0, 1, 1	Fault
7	5	4, 1, 5	1, 1, 1	Fault
8	2	1, 5, 2	0, 0, 1	Fault
9	1	1, 5, 2	1, 0, 1	Hit
10	3	2, 1, 3	1, 0, 1	Fault

Total page faults: 8

LFU hand trace

Step	Page	Frames	Frequency counts	Result
1	1	1	1	Fault
2	2	1, 2	1, 1	Fault
3	3	1, 2, 3	1, 1, 1	Fault
4	2	1, 2, 3	1, 2, 1	Hit
5	4	2, 3, 4	2, 1, 1	Fault
6	1	2, 4, 1	2, 1, 1	Fault
7	5	2, 1, 5	2, 1, 1	Fault
8	2	2, 1, 5	3, 1, 1	Hit
9	1	2, 1, 5	3, 2, 1	Hit
10	3	2, 1, 3	3, 2, 1	Fault

Total page faults: 7

Figure 2. Hand trace used to verify Clock and LFU, which have no single published fault count.

2.3 Reference patterns

F denotes the number of frames. Six generators produce reference strings with distinct locality, each seeded for repeatability.

Table 2. Reference patterns

Pattern	Generation	Locality tested
random	Uniform over 4F pages	No locality
zipf	8F pages, Zipf exponent 1.1, random page permutation	Frequency skew
working_set	10 phases; each has 0.75F hot pages (90 % of references), 10 % uniform over 16F pages	Temporal locality, shifting set
loop	Cyclic scan of 1.25F pages with 2 % random noise	Loop larger than memory
scan_mix	0.5F hot pages (50 % of references) interleaved with a never-repeating sequential scan	Scan pollution
phase_mix	8 equal segments, each drawn at random from the five patterns above	Behaviour change over time

2.4 Adaptive mechanisms

Adaptive-F (feature-based). Every window of W = 4F references it computes: working-set ratio (distinct pages / F), reuse ratio (1 − distinct / W), sequential fraction (share of reference pairs with page + 1), and skew (share of references that hit the top 20 % most-referenced pages). It then selects the eviction rule for the next window using the ordered rules in Table 3.

Table 3. Adaptive-F decision rules (first match wins)

Condition	Rule	Rationale
working-set ratio ≤ 0.9	LRU	Working set fits in memory; recency is safe
reuse < 0.1	FIFO	Pure scan; nothing helps, so use the cheapest rule
sequential ≥ 0.5 and reuse ≥ 0.3	MRU	Cyclic loop larger than memory defeats LRU

Condition	Rule	Rationale
skew ≥ 0.45	LFU	Strong frequency skew or scan pollution
reuse ≥ 0.5, otherwise	LRU, Clock	Good temporal locality → LRU; weak locality → cheap Clock

Adaptive-S (shadow caches). Ghost caches replay every candidate rule on the same reference stream. At the end of each window the real cache adopts the rule with the fewest faults in that window, but only if it beats the current rule by more than 5 % (hysteresis, to avoid thrashing between rules). This measures actual performance instead of assuming how features map to rules, at the price of about six times the work per reference.

2.5 Experimental setup

Three operating points were used: (N = 10 000, F = 16), (50 000, 32) and (100 000, 64). Every pattern, operating point and policy was run with seeds 0 to 9, and all policies saw identical reference strings. Adaptive thresholds were tuned only on separate seeds (1000 to 1003, N = 20 000, F = 32) so that the reported seeds are unseen. Significance is assessed with a paired Wilcoxon signed-rank test [5] on fault counts per seed, comparing each adaptive policy with the best required fixed policy (FIFO, LRU, LFU, Clock) for that pattern. With ten seeds, the smallest attainable two-sided p-value is 0.002. All runs were single-threaded on one machine.

Environment: 13th Gen Intel(R) Core(TM) i5-13420H processor, 16 GB RAM (15.65 GB reported), Microsoft Windows 11 (Build 26200), Python 3.13.7. All experiments were executed on the same machine using a single-threaded simulation.

3. Results

3.1 Hit ratio

Table 4. Mean hit ratio (%) at N = 100 000, F = 64 (10 seeds); best online policy per row shaded (ties all shaded)

Pattern	OPT	FIFO	LRU	LFU	Clock	MRU	Adapt-F	Adapt-S
random	59.6	25.0	25.1	25.0	25.0	25.0	25.0	25.0
zipf	81.3	60.7	66.5	73.3	65.2	17.1	73.3	73.0
working_set	91.5	75.4	87.3	18.0	84.2	8.3	87.3	87.4
loop	80.0	0.6	1.0	70.8	0.8	26.9	26.7	67.7
scan_mix	50.0	29.4	36.1	49.9	33.1	0.8	49.3	49.9
phase_mix	72.8	36.5	40.7	23.2	39.5	8.0	47.8	52.6

Results at N = 10 000 and 50 000 show the same overall pattern (Figure 4; full tables are in summary.csv), with small changes in rank between the two adaptive policies on phase_mix.

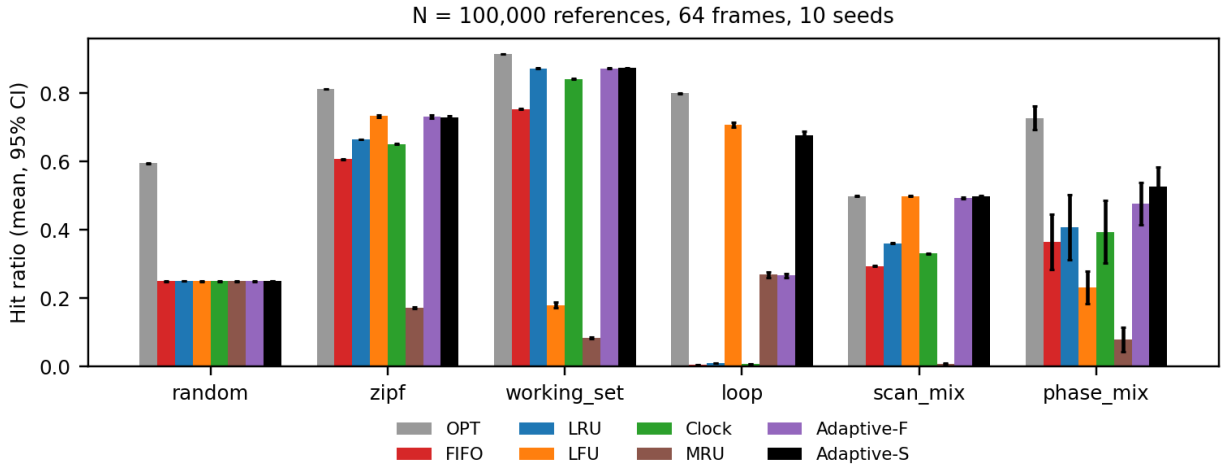


Figure 3. Hit ratio by pattern at N = 100 000 (error bars: 95 % confidence interval over 10 seeds).

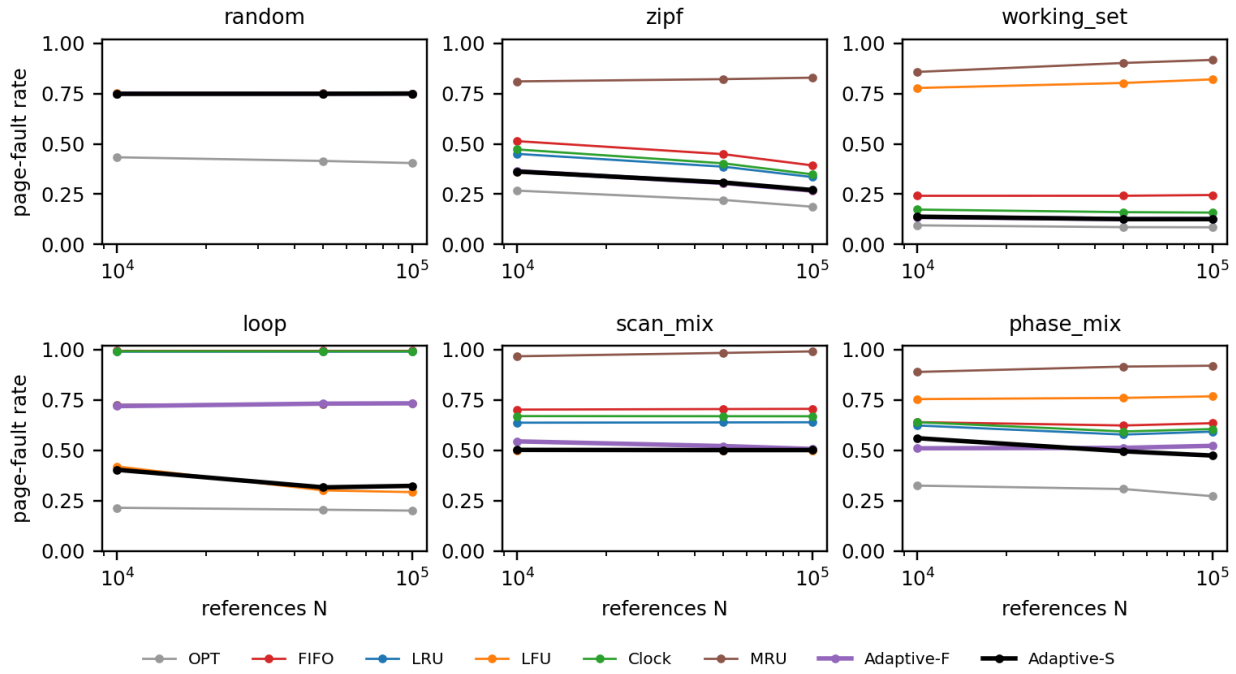


Figure 4. Page-fault rate as the workload grows from 10 000 to 100 000 references (adaptive policies drawn thicker).

3.2 Statistical comparison with fixed policies

Table 5. Change in mean page faults of the adaptive policy vs. a fixed policy (negative = fewer faults); Wilcoxon p in brackets

Pattern	N	Versus	Adaptive-F	Adaptive-S
phase_mix	10 000	LRU	−18.2 % (0.002)	−10.2 % (0.002)
phase_mix	50 000	LRU	−11.4 % (0.002)	−14.4 % (0.006)
phase_mix	100 000	LRU	−11.9 % (0.010)	−20.1 % (0.002)
loop	10 000	LFU	+71.6 % (0.002)	−3.9 % (0.275)
loop	50 000	LFU	+143.4 % (0.002)	+5.0 % (0.064)
loop	100 000	LFU	+151.1 % (0.002)	+10.6 % (0.004)
scan_mix	10 000	LFU	+8.6 % (0.002)	+0.2 % (0.002)
scan_mix	50 000	LFU	+4.1 % (0.002)	+0.08 % (0.002)
scan_mix	100 000	LFU	+1.2 % (0.002)	+0.08 % (0.002)
working_set	10 000	LRU	−1.2 % (0.020)	+0.2 % (0.383)
working_set	50 000	LRU	−0.6 % (0.035)	−0.8 % (0.002)
working_set	100 000	LRU	−0.4 % (0.037)	−0.9 % (0.002)

For zipf (best fixed: LFU) the adaptive policies differ from LFU by less than 1.5 % at every size ($p \geq 0.084$), and for random all policies differ by less than 0.2 %.

3.3 Scalability and overhead

Table 6. Mean simulation time per run in seconds (mean over six patterns)

N	OPT	FIFO	LRU	LFU	Clock	MRU	Adapt-F	Adapt-S
10 000	0.006	0.015	0.014	0.026	0.025	0.019	0.024	0.143
50 000	0.046	0.097	0.087	0.213	0.172	0.128	0.157	0.969
100 000	0.132	0.279	0.259	0.754	0.531	0.382	0.463	2.961

Adaptive-F costs about 1.8 times LRU at N = 100 000, because it adds one feature computation per window on top of the cache simulation. Adaptive-S costs about 11 times LRU, since it simulates five ghost caches. The simulator evicts by a linear scan over the frames, so time grows with both N and F (F grows with N across the three operating points); absolute times therefore describe this implementation, not a real kernel.

3.4 Adaptive behaviour on a changing workload

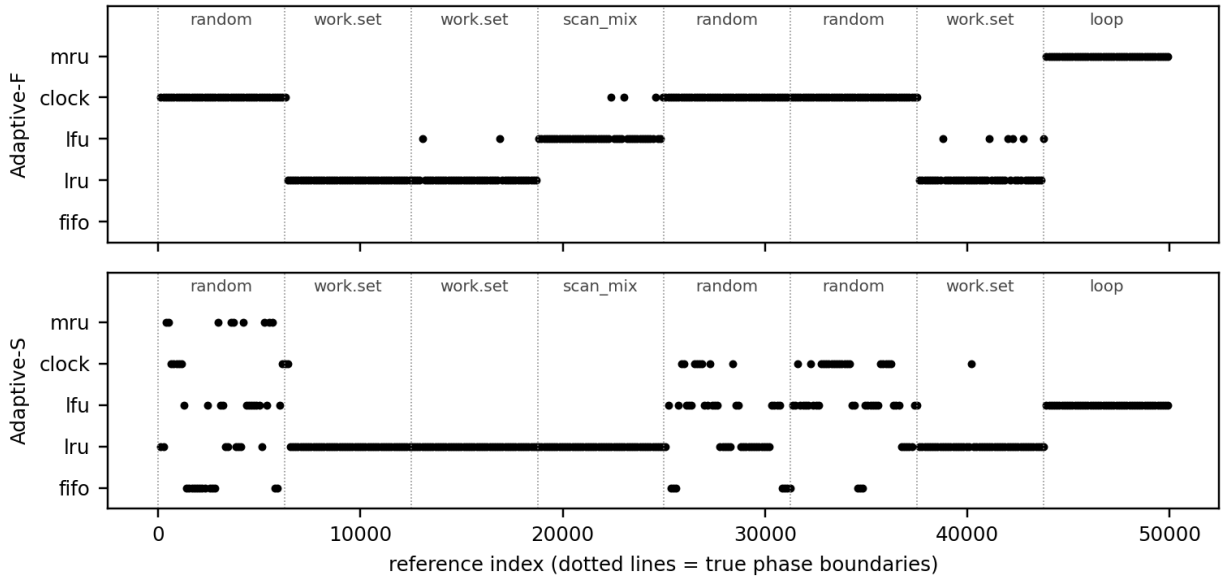


Figure 5. Rule chosen in each window on one phase_mix run (seed 0, N = 50 000, F = 32). Dotted lines mark true phase boundaries.

Adaptive-F's choices follow the phases closely: Clock on random segments, LRU on working-set segments, LFU on the scan_mix segment, and MRU on the final loop segment. On average it switched rule about 19 times per phase_mix run and 0.1 times on random. Adaptive-S also tracks the phases but is noisier on random segments (about 112 switches per random run, which does no harm because no rule is better there), and on this particular scan_mix segment it stayed with LRU. It chose LFU on the loop segment, which is why it handles loops far better than Adaptive-F.

```
PS C:\Users\shupr\Downloads\cs3_adaptive_paging\cs3_adaptive_paging> py analyze.py
MEAN HIT RATIO
 policy                OPT   FIFO    LRU    LFU  Clock    MRU  Adaptive-F  Adaptive-S
n_refs pattern
10000  loop          0.786  0.006  0.010  0.581  0.008  0.274       0.281       0.597
       phase_mix     0.676  0.360  0.377  0.246  0.360  0.111       0.490       0.440
       random        0.567  0.251  0.250  0.249  0.251  0.251       0.250       0.251
       scan_mix      0.499  0.298  0.363  0.499  0.330  0.033       0.456       0.498
       working_set   0.905  0.758  0.862  0.222  0.827  0.143       0.864       0.862
       zipf          0.733  0.487  0.550  0.638  0.528  0.190       0.636       0.638
50000  loop          0.795  0.006  0.010  0.699  0.008  0.269       0.268       0.684
       phase_mix     0.693  0.377  0.422  0.240  0.406  0.084       0.488       0.505
       random        0.585  0.251  0.251  0.251  0.251  0.249       0.251       0.251
       scan_mix      0.500  0.295  0.361  0.500  0.331  0.016       0.479       0.499
       working_set   0.914  0.758  0.873  0.197  0.840  0.098       0.874       0.874
       zipf          0.779  0.551  0.614  0.695  0.597  0.179       0.694       0.692
100000 loop          0.800  0.006  0.010  0.708  0.008  0.269       0.267       0.677
       phase_mix     0.728  0.365  0.407  0.232  0.395  0.080       0.478       0.526
       random        0.596  0.250  0.251  0.250  0.250  0.250       0.250       0.250
       scan_mix      0.500  0.294  0.361  0.499  0.331  0.008       0.493       0.499
       working_set   0.915  0.754  0.873  0.180  0.842  0.083       0.873       0.874
       zipf          0.813  0.607  0.665  0.733  0.652  0.171       0.733       0.730
```

```
PS C:\Users\shupr\Downloads\cs3_adaptive_paging\cs3_adaptive_paging> py analyze.py
SIGNIFICANCE vs best required fixed policy
     pattern  n_refs   adaptive versus  mean_fault_change_pct  wilcoxon_p
        loop   10000 Adaptive-F    LFU                 71.560       0.002
        loop   10000 Adaptive-S    LFU                 -3.899       0.275
        loop   50000 Adaptive-F    LFU                143.427       0.002
        loop   50000 Adaptive-S    LFU                  5.000       0.064
        loop  100000 Adaptive-F    LFU                151.106       0.002
        loop  100000 Adaptive-S    LFU                 10.589       0.004
  phase_mix   10000 Adaptive-F    LRU                -18.202       0.002
  phase_mix   10000 Adaptive-S    LRU                -10.160       0.002
  phase_mix   50000 Adaptive-F    LRU                -11.436       0.002
  phase_mix   50000 Adaptive-S    LRU                -14.392       0.006
  phase_mix  100000 Adaptive-F    LRU                -11.888       0.010
  phase_mix  100000 Adaptive-S    LRU                -20.061       0.002
     random   10000 Adaptive-F  Clock                  0.039       0.031
     random   10000 Adaptive-S  Clock                 -0.100       0.391
     random   50000 Adaptive-F  Clock                 -0.005       0.141
     random   50000 Adaptive-S  Clock                  0.026       0.922
     random  100000 Adaptive-F    LRU                  0.019       0.846
     random  100000 Adaptive-S    LRU                  0.052       0.375
   scan_mix   10000 Adaptive-F    LFU                  8.629       0.002
   scan_mix   10000 Adaptive-S    LFU                  0.202       0.002
   scan_mix   50000 Adaptive-F    LFU                  4.106       0.002
   scan_mix   50000 Adaptive-S    LFU                  0.078       0.002
   scan_mix  100000 Adaptive-F    LFU                  1.178       0.002
   scan_mix  100000 Adaptive-S    LFU                  0.075       0.002
working_set   10000 Adaptive-F    LRU                 -1.184       0.020
working_set   10000 Adaptive-S    LRU                  0.211       0.383
working_set   50000 Adaptive-F    LRU                 -0.612       0.035
working_set   50000 Adaptive-S    LRU                 -0.832       0.002
working_set  100000 Adaptive-F    LRU                 -0.350       0.037
working_set  100000 Adaptive-S    LRU                 -0.939       0.002
       zipf   10000 Adaptive-F    LFU                  0.403       0.750
       zipf   10000 Adaptive-S    LFU                 -0.028       1.000
       zipf   50000 Adaptive-F    LFU                  0.031       0.406
       zipf   50000 Adaptive-S    LFU                  0.785       0.492
       zipf  100000 Adaptive-F    LFU                  0.332       0.301
       zipf  100000 Adaptive-S    LFU                  1.419       0.084
```

Figure 6. Raw data and console output from the experiment run.

4. Discussion

Where adaptivity helps. Only phase_mix has no single best rule, and there both adaptive policies beat every fixed one: at N = 100 000, 47.8 % (Adaptive-F) and 52.6 % (Adaptive-S) hit ratio against 40.7 % for the best fixed policy, LRU (Table 4). On stationary patterns the adaptive policies match the best fixed policy within about 1 % on zipf and scan_mix and equal or slightly beat LRU on working_set, so the price of adaptivity on well-behaved workloads is small. On random, no policy can exceed the 25 % that the frame-to-page ratio allows, so all tie.

Where it fails. On the noisy loop, Adaptive-F identifies the loop correctly (Figure 5) but its fixed mapping "loop → MRU" is the wrong action: MRU reaches 26.7 % hits while LFU reaches 70.8 %, so Adaptive-F produces up to 151 % more faults than LFU. During tuning, mapping loops to LFU cut the loop fault rate from about 0.71 to 0.37 but raised the phase_mix fault rate from about 0.47 to 0.51, because LFU keeps stale high-count pages after a phase change. This shows that detecting the pattern is not enough; the rule-to-policy mapping must itself be right, and a hand-built mapping tuned on one workload mix is brittle. Adaptive-S avoids that assumption by measuring real faults, and is close to LFU on loops (+5 % to +11 % at larger sizes, significant only at N = 100 000).

Fixed-policy weaknesses confirmed. LRU, FIFO and Clock all score about 1 % on the loop, and LFU reaches only 18 % on working_set because counts from earlier phases pin obsolete pages. These are the classical weaknesses that motivate adaptivity.

Gap to optimal and cost. Averaged over all six patterns, Adaptive-F produces 76 to 90 % more faults than OPT and Adaptive-S 47 to 53 % more, depending on size. Part of this gap is unavoidable, since OPT sees the future (on random, OPT incurs about 46 % fewer faults than any online policy). Adaptive-S's extra accuracy costs about six times the per-reference work, which a real kernel would likely find too expensive; a cheaper variant could run ghost caches on a sampled subset of references. In a real system, however, one avoided fault saves milliseconds of I/O, so simulator time is a poor proxy for the true cost-benefit.

Threats to validity. (i) The workloads are synthetic and the thresholds were tuned on the same generators, so results may not transfer to real application traces. (ii) Ten seeds give limited statistical power, and $p = 0.002$ is the minimum attainable. (iii) Our LFU counts references only while a page is resident; other LFU variants behave differently. (iv) Timing was measured on one machine and excludes I/O cost, MMU effects and multiprogramming. (v) Window size (4F) and the 5 % hysteresis were fixed after small tuning runs and were not systematically explored.

5. Conclusion

A locality-aware page-replacement policy is worthwhile when program behaviour changes over time: both adaptive designs cut page faults by about 10 to 20 % against the best fixed policy on a mixed workload, with statistical significance, while staying near the best fixed policy on most stationary patterns. The study also shows the limit of hand-written rules: Adaptive-F fails on noisy loops because its mapping from pattern to rule is wrong, whereas the measurement-driven Adaptive-S is more robust but about six times more expensive. Future work includes testing on real memory traces, sampled shadow caches, learned or self-tuning rule mappings (for example ARC-style policies [4]), and systematic window-size studies.

6. Individual Contributions

Member	Reg. no.	Specific individual contribution
Shupraja N	24BCE0560	Primary implementation of the algorithms and adaptive mechanisms, code integration, validation, experiment execution, result analysis, visualization and report preparation.
Aishwarya Vaidyanathan	24BCE2683	Contribution to algorithm research, code development and review, analysis of workload behaviour, interpretation of experimental results and report refinement.

References

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018 (virtual-memory chapter).

[2] L. A. Belady, R. A. Nelson and G. S. Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," Commun. ACM, vol. 12, no. 6, pp. 349–353, 1969.

[3] P. J. Denning, "The working set model for program behavior," Commun. ACM, vol. 11, no. 5, pp. 323–333, 1968.

[4] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in Proc. USENIX FAST, 2003.

[5] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.